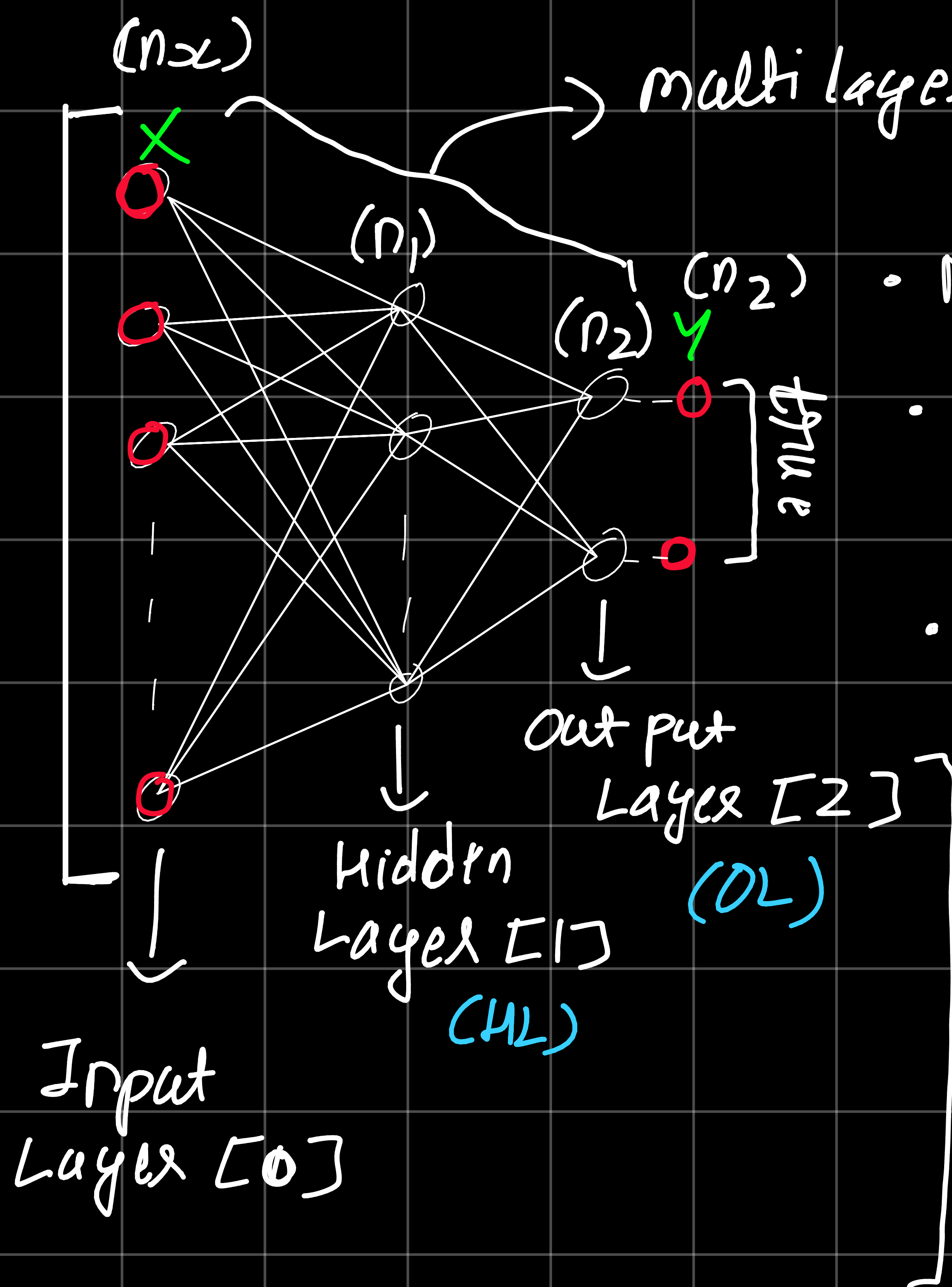


A Simple Neural Network (From Scratch)



- Neuron: Holds Value.
- Weight: strength of connection between two neurons
- Bias: Additional tuning

L : no. of layers in network

where $L=1$: first HL

$L=L$: OL

- $a_j^{[L]}$: j^{th} neuron's activation in L^{th} layer
- $z_j^{[L]}$: j^{th} neuron's pre-act. in L^{th} layer.
- $w_{jk}^{[L]}$: weight of connection between j^{th} neuron in layer L , & k^{th} neuron in layer $L-1$.
/ $w_{jk}^{[L]}$
- $b_j^{[L]}$: bias of j^{th} neuron in layer L

Exl: Pre-Act. value of neuron $a_j^{[L]}$:

single
neuron, one example

$$\begin{aligned} z_j^{[L]} &= w_{j0}^{[L]} a_0^{[L-1]} + w_{j1}^{[L]} a_1^{[L-1]} + \dots \\ &\quad w_{jk}^{[L]} a_k^{[L-1]} + b_j^{[L]} \\ &= \sum_k w_{jk}^{[L]} a_k^{[L-1]} + b_j^{[L]} \end{aligned}$$

(with batch size: m)
for every neuron in layer L , this can be
written as:

W^L : wt. matrix of L

$$\begin{bmatrix} w_{11}^L & w_{12}^L & \dots & w_{1n_{L-1}}^L \\ w_{21}^L & w_{22}^L & \dots & w_{2n_{L-1}}^L \\ \vdots & \vdots & \ddots & \vdots \\ w_{n_L 1}^L & w_{n_L 2}^L & \dots & w_{n_L n_{L-1}}^L \end{bmatrix}$$

$n_L \times n_{L-1}$
neurons in current layer
neurons in prev layer

A^{L-1} : Neuron Activations

$$\begin{bmatrix} a_1^{[L-1(1)]} & \dots & a_1^{[L-1(m)]} \\ a_2^{[L-1(1)]} & \dots & a_2^{[L-1(m)]} \\ \vdots & \ddots & \vdots \\ a_{n_{L-1}}^{[L-1(1)]} & \dots & a_{n_{L-1}}^{[L-1(m)]} \end{bmatrix}$$

$n_{L-1} \times m$
batch size

each column is training example.

B^L : bias matrix of L

$$\begin{bmatrix} b_1^L \\ b_2^L \\ \vdots \\ b_{n_L}^L \end{bmatrix}$$

$n_L \times 1$
broadcast casted to $(n_L \times m)$

pre-Activation matrix of layer L

$$+ = z^L$$

$$z_{(n_L \times m)}^L = W_{(n_L \times n_{L-1})}^L A_{(n_{L-1} \times m)}^{L-1} + B_{(n_L \times 1)}^L$$

Forward Propagation

$$A^L = g(z^L)$$

Activation function

Use Act. fn. to prevent linear rel. b/w layers & add complexity to capture more features than a linear reggers.

Notation:

- 1. Capital Letters: Matrices (Macro)
- 2. Small Letters: Scalars (Micro)

A Dummy Network Architecture

- $X \in \mathbb{R}^{n_L \times m}$ (input layer)
- L_1 • $A^{L1}, Z^{L1} \in \mathbb{R}^{n_1 \times m}$ (hidden layer)
 - $W^{L1} \in \mathbb{R}^{n_1 \times n_L}$
 - $b^{L1} \in \mathbb{R}^{n_1 \times 1}$
- L_2 • $A^{L2}, Z^{L2} \in \mathbb{R}^{n_2 \times m}$ (output layer)
 - $W^{L2} \in \mathbb{R}^{n_2 \times n_1}$
 - $b^{L2} \in \mathbb{R}^{n_2 \times 1}$
 - $Y \in \mathbb{R}^{n_2 \times m}$

Activations:

- $L_1 \rightarrow \text{ReLU} : \max(0, z)$
- $L_2 \rightarrow \text{Softmax} : \left\{ \frac{e^{z_k^{L2}(i)}}{\sum_{k=1}^{n_L} e^{z_k^{L2}}} \right\} = C_k^{(i)}$

- Loss: Cross Entropy

$$\left\{ \begin{aligned} C^{(i)} &= - \sum_{j=1}^{n_2} y_j^{(i)} \log(a_j^{L2}(i)) \\ C &= \frac{1}{m} \sum_{i=1}^m C^{(i)} \rightarrow \text{Batch (Avg.) Loss} \end{aligned} \right.$$

1. Forward Propagation:

$$Z^{[1]} = W^{[1]} X + B^{[1]}$$

$$A^{[1]} = g(Z^{[1]}) ; g \approx \text{ReLU}$$

$$Z^{[2]} = W^{[2]} A^{[1]} + B^{[2]}$$

$$A^{[2]} = \text{softmax}(Z^{[2]})$$

2. Loss: Y _(n₂ × m) column is one-hot encoded
i.e if element $c(i)$ matches with
class $c(i)$ then $y_j^{(i)} = 1$
where $j = c(i)$ else 0.
 $\{ A^{[2]} \}$ _(n₂ × m) has probabilities b/w 0 & 1

By Shannon's Information Theory:

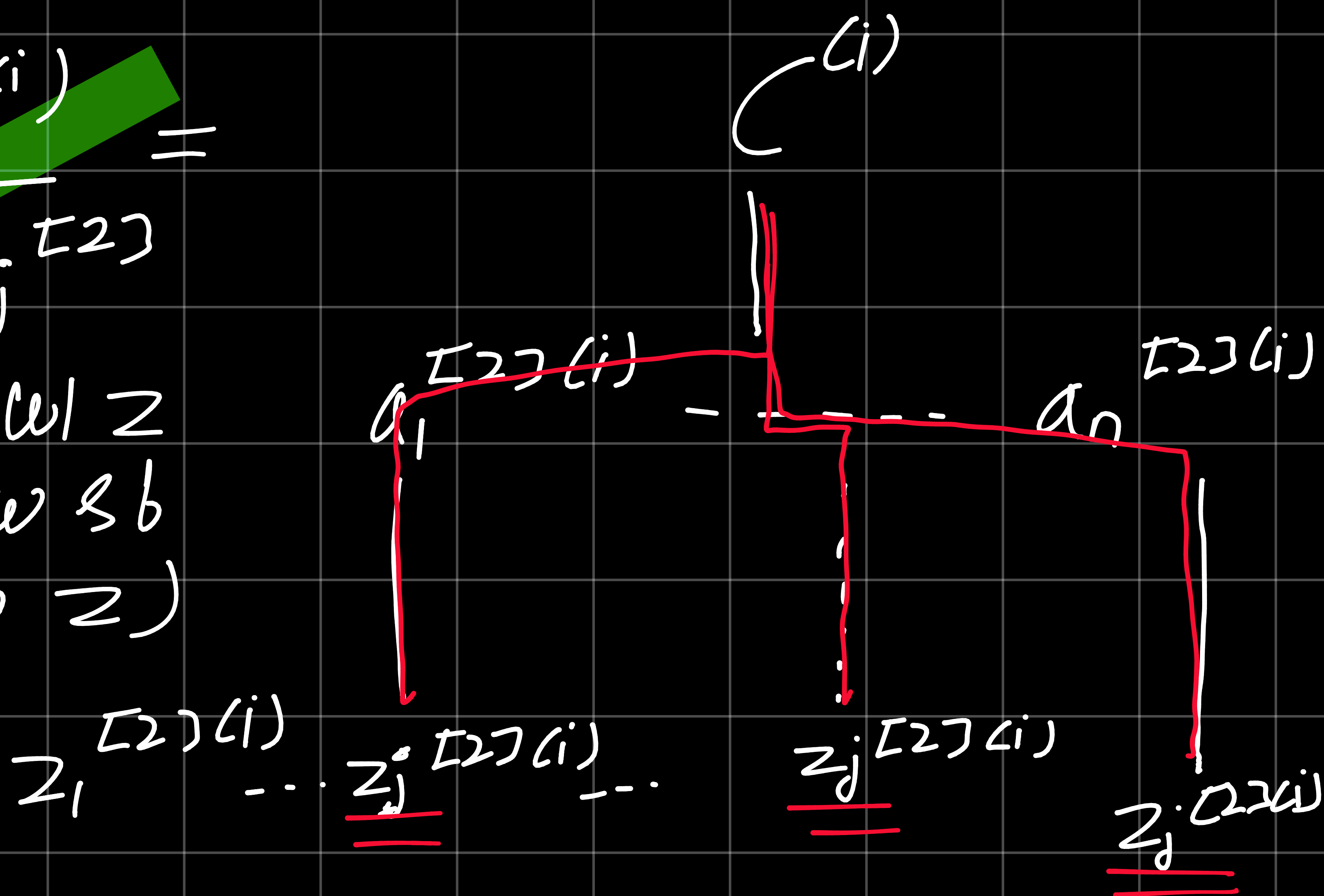
$$\checkmark \quad \text{loss in example } \underline{\underline{i}} = - \sum_{j=1}^{n_2} y_j^{(i)} \log(a_j^{[2]}(i))$$

3. Back Propagation:

"How should z change weights & biases to minimize loss?"

$$i) \frac{\partial C^{(i)}}{\partial z_j^{[2]}} =$$

* (find dep. w/ z first as w & b depend on z)



Accounting for All paths:

$$\frac{\partial C^{(i)}}{\partial z_j^{[2]}} = \sum_{k=1}^{n_2-1} \frac{\partial C^{(i)}}{\partial a_k^{[2]}} \frac{\partial a_k^{[2]}}{\partial z_j^{[2]}}$$

* when $k=j$: Path is direct ($a_1 \rightarrow z_1$)

* when $k \neq j$: Path is indirect ($a_2 \rightarrow z_1$)

We know, from $C^{(i)}$:

$$\frac{\partial C^{(i)}}{\partial a_k^{[2]}} = - \frac{y_k^{(i)}}{a_k^{[2]^{[2]}}}$$

$$\left\{ \text{And } a_k^{[2]}(i) = \frac{e^{z_k^{[2]}(i)}}{S} \right.$$

$$\left\{ \text{where } S = \sum_{k=1}^{n_2} e^{z_k^{[2]}(i)} \right.$$

(case - I) $k=j$

Quotient Rule

$$\left\{ \frac{da_j^{[2]}(i)}{dz_j^{[2]}(i)} = \frac{e^{z_j^{[2]}(i)}}{S} - \frac{(e^{z_j^{[2]}(i)})^2}{S^2} \right.$$

$$= \frac{a_j^{[2]}(i) (1 - a_j^{[2]}(i))}{S}$$

$$\left\{ \frac{\partial C^{(i)}}{da_j^{[2]}(i)} = \frac{-g_j}{a_j^{[2]}(i)} \right.$$

(case - II) $k \neq j$

Quo. Rule

$$\left\{ \frac{da_k^{[2]}(i)}{dz_j^{[2]}(i)} = - \frac{e^{z_k^{[2]}(i)} e^{z_j^{[2]}(i)}}{S^2} \right.$$

$$= - \frac{a_k^{[2]}(i) a_j^{[2]}(i)}{S}$$

$$\left\{ \frac{\partial C^{(i)}}{da_k^{[2]}(i)} = \frac{-g_k}{a_k^{[2]}(i)} \right.$$

$\Rightarrow$ Splitting Sum:

$$-\frac{y_j}{a_j^{[2](i)}} \quad \overbrace{(a_j^{[2](i)})(1 - a_j^{[2](i)})}^{k=j}$$

$$+ \sum_{\substack{k=1 \\ k \neq j}}^{n_2} \frac{-y_k}{a_k^{[2](i)}} \quad \underbrace{(-a_k^{[2](i)} a_j^{[2](i)})}_{k \neq j}$$

$$= -y_j + a_j^{[2](i)} y_j + a_j^{[2](i)} \sum_{\substack{k=1 \\ k \neq j}}^{n_2} y_k$$

$$= -y_j + a_j^{[2](i)} y_j + a_j^{[2](i)} \underbrace{(1 - y_j)}$$

Since $\sum_{k=1}^{n_2} y_k = 1$
Due to one-hot enc.

$$\Rightarrow \boxed{a_j^{[2](i)} - y_j} = \frac{\delta^{(i)}}{\delta z_j^{[2](i)}} = \delta z_j^{[2](i)}$$

$$\therefore \delta z_j^{[2](i)} = a_j^{[2](i)} - y_j$$

$$\text{Vectorization: } \boxed{\delta Z^{[2]} = A^{[2]} - Y}$$

(j rows, i cols)

Takes care of ALL paths

$$\text{ii)} \quad \frac{\partial C^{(i)}}{\partial w_{jk}^{[2]}} = \frac{\partial C^{(i)}}{\partial z_j^{[2]}} \frac{\partial z_j^{[2]}}{\partial w_{jk}^{[2]}}$$

$\frac{\partial z_j^{[2]}}{\partial w_{jk}^{[2]}}$

$$z_j^{[2](i)} = w_{jk}^{[2]} a_k^{[1](i)} + b_j^{[2]}$$

$$\therefore \frac{\partial z_j^{[2](i)}}{\partial w_{jk}^{[2]}} = a_k^{[1](i)}$$

$$\therefore C = \frac{1}{m} \sum_{i=1}^m C_i$$

$$\left\{ \frac{\partial C}{\partial w_{jk}^{[2]}} \right\} = \frac{1}{m} \sum_{i=1}^m \frac{\partial z_j^{[2](i)}}{\partial w_{jk}^{[2]}} a_k^{[1](i)}$$

$\begin{matrix} j^{\text{th}} \text{ neuron,} \\ \uparrow \\ i^{\text{th}} \text{ col.} \end{matrix}$

$\begin{matrix} k^{\text{th}} \text{ neuron,} \\ \downarrow \\ i^{\text{th}} \text{ row} \end{matrix}$

$\left\{ \frac{\partial C}{\partial w_{jk}^{[2]}} \right\}$ element (j,k) of $\partial W^{[2]}$

Vectorization:
(j rows,
k cols.)

$$\partial W^{[2]} = \frac{1}{m} \partial Z^{[2]} (A^{[1]})^T$$

$$\text{iii)} \quad \frac{\partial C^{(i)}}{\partial b_j^{[2]}} = \underbrace{\frac{\partial C^{(i)}}{\partial z_j^{[2]}}}_{d z_j^{[2]}(i)} \underbrace{\frac{\partial z_j^{[2]}(i)}{\partial b_j^{[2]}}}_{= 1^* \text{ from Above eqn}}$$

$$\therefore \frac{\partial C}{\partial b_j^{[2]}} = \frac{1}{m} \sum_{i=1}^m d z_j^{[2]}(i)$$

Vectorization:
(j rows, i cols.)

$$dB^{[2]} = \frac{1}{m} \sum_{i=1}^m dZ^{[2]}(i)$$

For Hidden Layers ($dZ^{[1]}$, $dW^{[1]}$, $dB^{[1]}$)

$$\text{i)} \quad \frac{\partial C^{(i)}}{\partial z_k^{[1]}(i)} = \left\{ \begin{array}{c} a_1 \\ \vdots \\ a_k \\ \vdots \\ a_n \end{array} \right\} \begin{array}{c} \downarrow \\ z^{[2]} \\ \downarrow \\ z_k \\ \downarrow \\ z_n \end{array}$$

(Note: $a_k^{[1]}$ has ReLU not softmax)

* Summing over Paths:

$$\sum_{j=1}^{n_2} \underbrace{\frac{\partial C^{(i)}}{\partial z_j^{[2]}(i)} \frac{\partial z_j^{[2]}(i)}{\partial a_k^{[1]}(i)}}_{dA_k^{[1]}(i)} \frac{\partial a_k^{[1]}(i)}{\partial z_k^{[1]}(i)}$$

$$\frac{d^{(i)}}{dz_j^{[2](i)}} = dz_j^{[2](i)} \quad \left. \vphantom{\frac{d^{(i)}}{dz_j^{[2](i)}}} \right\} dA_k^{[1](i)}$$

$$\frac{dz_j^{[2](i)}}{dA_k^{[1](i)}} = w_{jk}^{[23]}$$

$$\frac{dA_k^{[1](i)}}{dz_k^{[1](i)}} = g'(z_k^{[1](i)})$$

$$= \sum_{j=1}^{n_2} dz_j^{[2](i)} w_{jk}^{[23]} g'(z_k^{[1](i)})$$

$$= g'(z_k^{[1](i)}) \sum_{j=1}^{n_2} dz_j^{[2](i)} w_{jk}^{[23]}$$

$$= g'(z_k^{[1](i)}) \sum_{j=1}^{n_2} w_{jk}^{[23]} dz_j^{[2](i)}$$

Vectorisation (i cols, k rows):

$$dZ^{[1]} = (w_{jk}^{[23]})^T dZ^{[2]} \odot g'(Z^{[1]})$$

Also called: $dA^{[1]}$

$$ii) \frac{dL^{(i)}}{dw_{jk}^{L1}} = \frac{dL^{(i)}}{dz_j^{L1(i)}} \frac{dz_j^{L1(i)}}{dw_{jk}^{L1}}$$

$\underbrace{\frac{dL^{(i)}}{dz_j^{L1(i)}}}_{dz_j^{L1(i)}} \quad \underbrace{\frac{dz_j^{L1(i)}}{dw_{jk}^{L1}}}_{a_k^{L0(i)}}$

$$\therefore \frac{dL}{dw_{jk}^{L1}} = \frac{1}{m} \sum_{i=1}^m dz_j^{L1(i)} a_k^{L0(i)}$$

$$dW^{L1} = \frac{1}{m} dZ^{L1} (A^{L0})^T$$

$$\boxed{dW^{L1} = \frac{1}{m} dZ^{L1} X^T} \quad \text{Vectorization} =$$

$$iii) \frac{dL^{(i)}}{db_j^{L1}} = \frac{dL^{(i)}}{dz_j^{L1(i)}} \frac{dz_j^{L1(i)}}{db_j^{L1}}$$

$\underbrace{\frac{dL^{(i)}}{dz_j^{L1(i)}}}_{dz_j^{L1(i)}} \quad \underbrace{\frac{dz_j^{L1(i)}}{db_j^{L1}}}_{=1}$

$$\therefore \frac{dL}{db_j^{L1}} = \frac{1}{m} \sum_{i=1}^m dz_j^{L1(i)}$$

$$\boxed{dB^{L1} = \frac{1}{m} \sum_{i=1}^m dz^{L1(i)}} \quad \text{(Vectorization)}$$

Generalizing: (Softmax at L) + (Cross Entropy Loss)

$$\star dZ^{[L]} = A^{[L]} - Y \quad \} L: \text{last layer}$$

$(n_L \times m)$

$$\star dZ^{[L]} = (W^{[L+1]})^T \overbrace{Z^{[L+1]}}^{dA^{[L]}} \odot g'(Z^{[L]})$$

$(n_L \times m)$

$L \in \{1, 2, 3, \dots, L-1\}$

$$\star dB^{[L]} = \frac{1}{m} \sum_{i=1}^m dZ^{[L]}$$

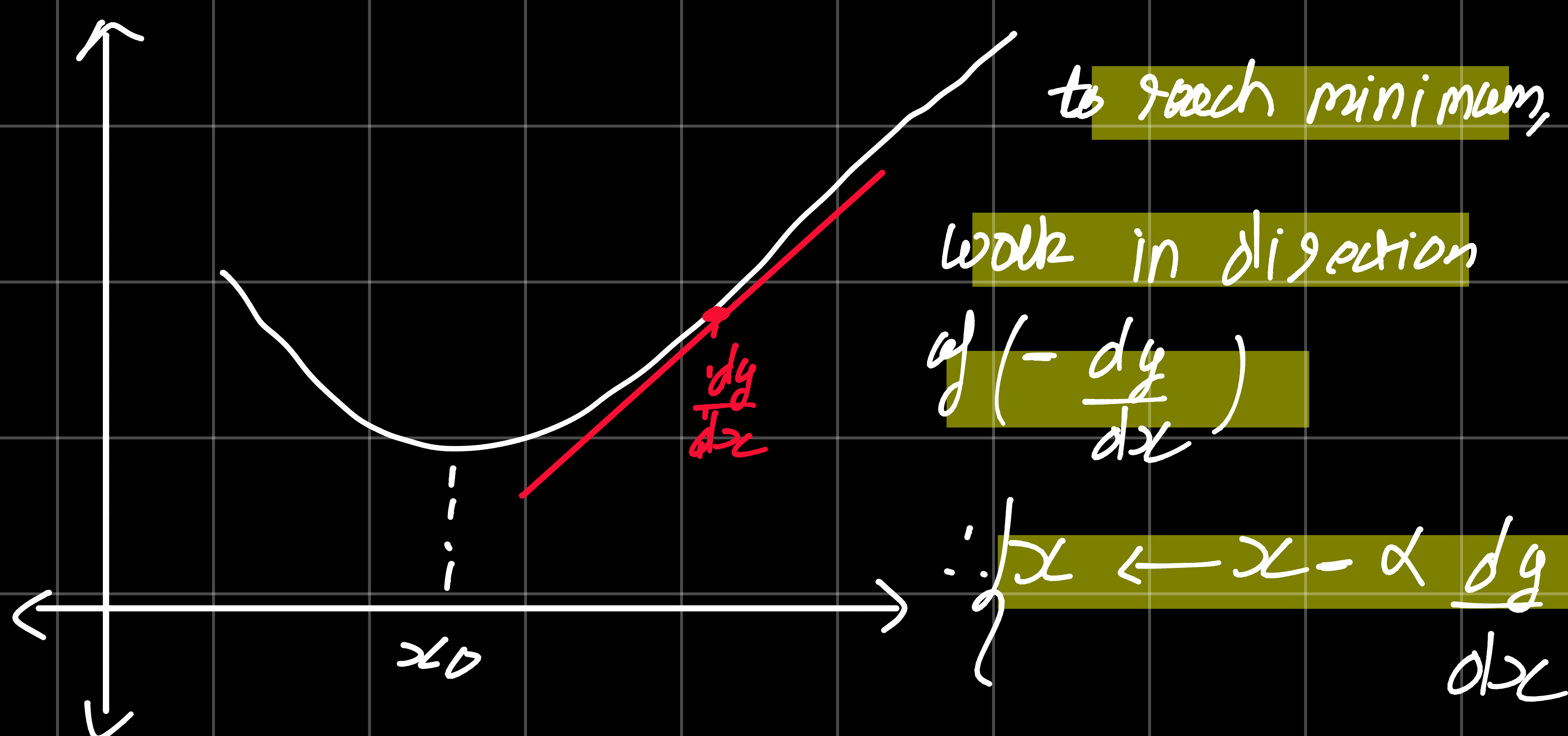
$(n_L \times 1)$

$$\star dW^{[L]} = \frac{1}{m} dZ^{[L]} (A^{[L-1]})^T$$

$(n_L \times n_{L-1})$

$(L > 0)$

Gradient Descent:

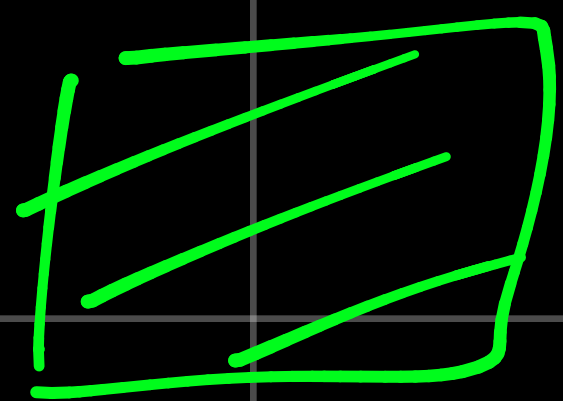


- where α is hyperparameter. / l.s.
- repeat until point stabilizes. (min)

for LS:

- $B^{[L]} = B^{[L]} - \alpha dB^{[L]}$
- $W^{[L]} = W^{[L]} - \alpha dW^{[L]}$
- Repeat until loss is minimized.
- Model is trained!

Q.E.D.



=> Code Architecture:

- nn /

- layers.py

- Dense

- W, B, dW, dB, A^{L-1}, Z

- MP init

- forward, backward

- ReLU

- Z, fW, bW

- Softmax Cross Entropy

- A, Y, fwd, bwd

- optimizer.py

- SGD

- layers, $L_n(\alpha)$

- step / update W, B

- model.py

- Sequential

- layers, loss layer, optimizer

- fwd, fit, predict

- init.py

